

Sistema de Inspeção de Equipamentos Suzano

Guia Completo de Implantação e Operação

Versão: 1.0

Data: Junho 2025

Autor: Manus AI

Empresa: Suzano S.A.

Sumário Executivo

O Sistema de Inspeção de Equipamentos da Suzano representa uma solução tecnológica moderna e abrangente desenvolvida para otimizar e digitalizar os processos de inspeção de equipamentos industriais. Este sistema foi projetado para resolver os desafios identificados no Desafio de Inovação 2025 da Suzano, oferecendo uma plataforma robusta que elimina erros de identificação de equipamentos, garante a precisão na coleta de dados e permite o armazenamento seguro de informações multimídia.

A arquitetura do sistema baseia-se em tecnologias cloud-native da Amazon Web Services (AWS), utilizando Java Spring Boot para o backend, React para a interface web e uma infraestrutura completamente automatizada via Terraform. Esta abordagem garante escalabilidade, confiabilidade e facilidade de manutenção, atendendo aos rigorosos padrões de qualidade e segurança exigidos pela indústria de papel e celulose.

O sistema oferece funcionalidades avançadas como identificação precisa de equipamentos via QR Code, captura automática de localização GPS, formulários de inspeção digitais com checklists customizáveis, armazenamento seguro de fotos e vídeos, geração automática de relatórios e alertas em tempo real para situações

críticas. Todas essas funcionalidades são integradas em uma interface intuitiva que pode ser acessada tanto via aplicativo móvel quanto através de navegadores web.

1. Introdução e Contexto

1.1 Visão Geral do Projeto

A Suzano, como uma das maiores empresas de papel e celulose do mundo, opera com milhares de equipamentos críticos distribuídos em suas unidades industriais. A manutenção preventiva e as inspeções regulares desses equipamentos são fundamentais para garantir a continuidade operacional, a segurança dos trabalhadores e a eficiência dos processos produtivos.

Tradicionalmente, as inspeções eram realizadas utilizando formulários em papel ou planilhas eletrônicas, métodos que apresentavam limitações significativas em termos de precisão, rastreabilidade e integração com sistemas corporativos. O Desafio de Inovação 2025 da Suzano identificou a necessidade de uma solução digital que pudesse superar essas limitações e proporcionar um salto qualitativo nos processos de inspeção.

1.2 Objetivos do Sistema

O Sistema de Inspeção de Equipamentos foi desenvolvido com os seguintes objetivos principais:

Precisão na Identificação: Eliminar completamente os erros de identificação de equipamentos através da implementação de códigos QR únicos e verificação automática de localização GPS. Esta funcionalidade garante que cada inspeção seja associada corretamente ao equipamento específico, eliminando confusões que poderiam levar a falhas de manutenção ou decisões baseadas em dados incorretos.

Digitalização Completa: Substituir formulários em papel por interfaces digitais intuitivas que permitem a coleta estruturada de dados, incluindo texto, imagens, vídeos e medições. A digitalização não apenas melhora a qualidade dos dados coletados, mas também facilita o armazenamento, busca e análise posterior das informações.

Rastreabilidade Total: Implementar um sistema de auditoria completo que registra automaticamente quem realizou cada inspeção, quando foi realizada, onde ocorreu e quais foram os resultados. Esta rastreabilidade é essencial para atender requisitos regulatórios e para análises de tendências históricas.

Integração Corporativa: Desenvolver APIs robustas que permitem a integração com sistemas existentes da Suzano, como ERP, sistemas de manutenção (CMMS) e plataformas de business intelligence. Esta integração garante que os dados de inspeção sejam automaticamente incorporados aos fluxos de trabalho existentes.

Mobilidade e Acessibilidade: Criar uma solução que funciona tanto em dispositivos móveis quanto em computadores desktop, permitindo que inspetores realizem seu trabalho diretamente no campo enquanto supervisores e gestores acompanham o progresso em tempo real através de dashboards web.

1.3 Benefícios Esperados

A implementação do sistema deve gerar benefícios tangíveis em múltiplas dimensões:

Operacionais: Redução significativa no tempo necessário para completar inspeções, eliminação de retrabalho causado por erros de identificação, melhoria na qualidade dos dados coletados e aumento na frequência de inspeções realizadas dentro dos prazos estabelecidos.

Financeiros: Diminuição dos custos operacionais através da redução de papel, eliminação de digitação manual de dados, redução de paradas não programadas devido a melhor detecção precoce de problemas e otimização dos cronogramas de manutenção preventiva.

Qualidade e Segurança: Melhoria na detecção precoce de anomalias através de checklists padronizados e obrigatórios, redução de riscos de segurança através de inspeções mais frequentes e precisas, e melhoria na conformidade com normas regulatórias através de documentação automática e completa.

Estratégicos: Criação de uma base de dados histórica robusta que permite análises preditivas, identificação de padrões de falhas, otimização de estratégias de manutenção e suporte a decisões baseadas em dados para investimentos em novos equipamentos ou modernizações.

2. Arquitetura do Sistema

2.1 Visão Geral da Arquitetura

O Sistema de Inspeção de Equipamentos da Suzano foi projetado seguindo os princípios de arquitetura cloud-native, microserviços e infraestrutura como código. A solução utiliza uma abordagem de três camadas (three-tier architecture) distribuída na nuvem AWS, garantindo alta disponibilidade, escalabilidade e segurança.

A arquitetura é composta pelos seguintes componentes principais:

Camada de Apresentação: Interface web responsiva desenvolvida em React que oferece uma experiência de usuário moderna e intuitiva. Esta camada é servida através do Amazon CloudFront para garantir baixa latência global e alta disponibilidade. A interface suporta tanto dispositivos desktop quanto móveis, adaptando-se automaticamente ao tamanho da tela e às capacidades do dispositivo.

Camada de Aplicação: Backend desenvolvido em Java Spring Boot executando em containers Docker no Amazon Elastic Container Service (ECS). Esta camada implementa toda a lógica de negócio, APIs RESTful, autenticação e autorização, processamento de dados e integração com serviços externos. O uso de containers garante consistência entre ambientes de desenvolvimento, teste e produção.

Camada de Dados: Banco de dados PostgreSQL gerenciado pelo Amazon RDS com configuração Multi-AZ para alta disponibilidade. O armazenamento de arquivos multimídia é realizado no Amazon S3 com diferentes classes de armazenamento para otimização de custos. Todos os dados são criptografados em repouso e em trânsito.

2.2 Componentes da Infraestrutura AWS

2.2.1 Rede e Conectividade

A infraestrutura de rede foi projetada seguindo as melhores práticas de segurança da AWS, implementando uma Virtual Private Cloud (VPC) dedicada com sub-redes públicas e privadas distribuídas em múltiplas zonas de disponibilidade.

Virtual Private Cloud (VPC): Uma VPC dedicada com CIDR 10.0.0.0/16 fornece isolamento completo da rede, permitindo controle total sobre o ambiente de rede

virtual. A VPC está configurada com DNS resolution e DNS hostnames habilitados para suportar a resolução de nomes internos.

Sub-redes Públicas: Duas sub-redes públicas (10.0.1.0/24 e 10.0.2.0/24) distribuídas em zonas de disponibilidade diferentes hospedam o Application Load Balancer e os NAT Gateways. Estas sub-redes têm acesso direto à internet através do Internet Gateway.

Sub-redes Privadas: Duas sub-redes privadas (10.0.10.0/24 e 10.0.20.0/24) hospedam os containers da aplicação e o banco de dados RDS. O tráfego de saída dessas sub-redes é roteado através dos NAT Gateways para garantir que os recursos não sejam diretamente acessíveis da internet.

Sub-redes de Banco de Dados: Duas sub-redes adicionais (10.0.30.0/24 e 10.0.40.0/24) são dedicadas exclusivamente ao banco de dados RDS, proporcionando uma camada adicional de isolamento e segurança.

2.2.2 Computação e Containers

Amazon ECS (Elastic Container Service): O serviço de orquestração de containers gerencia a execução da aplicação backend. O cluster ECS está configurado com Fargate como tipo de lançamento, eliminando a necessidade de gerenciar instâncias EC2 subjacentes.

Task Definition: Define a configuração dos containers, incluindo imagem Docker, recursos de CPU e memória, variáveis de ambiente, secrets e configurações de rede. A task definition especifica 512 CPU units e 1024 MB de memória, valores otimizados para o workload da aplicação.

ECS Service: Garante que o número desejado de tasks esteja sempre em execução, implementando health checks e substituição automática de containers com falha. O serviço está configurado com deployment configuration que permite atualizações rolling sem downtime.

Auto Scaling: Políticas de auto scaling baseadas em métricas de CPU e memória garantem que a aplicação possa lidar com variações de carga automaticamente, escalando horizontalmente conforme necessário.

2.2.3 Balanceamento de Carga e Distribuição

Application Load Balancer (ALB): Distribui o tráfego HTTP/HTTPS entre as instâncias da aplicação, implementando health checks avançados e roteamento baseado em path. O ALB está configurado com listeners para HTTP (porta 80) e HTTPS (porta 443), com redirecionamento automático de HTTP para HTTPS.

Target Groups: Grupos de destino configurados com health checks que verificam o endpoint /atuador/health da aplicação Spring Boot. Os health checks são realizados a cada 30 segundos com timeout de 5 segundos e threshold de 2 verificações consecutivas bem-sucedidas para considerar um target saudável.

SSL/TLS: Certificados SSL gerenciados pelo AWS Certificate Manager (ACM) garantem criptografia em trânsito. Os certificados são automaticamente renovados pelo ACM, eliminando a necessidade de gerenciamento manual.

2.2.4 Armazenamento de Dados

Amazon RDS PostgreSQL: Banco de dados relacional gerenciado configurado com Multi-AZ deployment para alta disponibilidade. A instância principal está na zona de disponibilidade us-east-1a com uma réplica standby síncrona em us-east-1b.

Configurações de Performance: A instância RDS está configurada com Performance Insights habilitado para monitoramento detalhado de performance. O parameter group customizado otimiza configurações específicas para o workload da aplicação.

Backup e Recovery: Backups automatizados são realizados diariamente com retenção de 7 dias. O backup window está configurado para 03:00-04:00 UTC para minimizar impacto na performance durante horários de pico.

Amazon S3: Múltiplos buckets S3 são utilizados para diferentes propósitos: - Bucket de mídia para armazenamento de fotos e vídeos de inspeções - Bucket de logs para armazenamento de logs do ALB - Bucket de backups para armazenamento de backups da aplicação

2.2.5 Segurança e Controle de Acesso

Security Groups: Grupos de segurança implementam firewalls virtuais que controlam o tráfego de entrada e saída: - ALB Security Group: Permite tráfego HTTP/HTTPS da internet - ECS Security Group: Permite tráfego apenas do ALB na porta da aplicação - RDS Security Group: Permite tráfego apenas dos containers ECS na porta 5432

IAM Roles e Policies: Roles IAM específicas são criadas para cada componente: - ECS Task Execution Role: Permite que o ECS execute tasks e acesse ECR e CloudWatch - ECS Task Role: Permite que a aplicação acesse S3 e Secrets Manager - RDS Enhanced Monitoring Role: Permite monitoramento detalhado do RDS

AWS Secrets Manager: Armazena credenciais sensíveis como senhas do banco de dados de forma segura, com rotação automática configurada.

2.3 Fluxo de Dados e Comunicação

O fluxo de dados no sistema segue um padrão bem definido que garante segurança, performance e rastreabilidade:

Autenticação e Autorização: Usuários acessam o sistema através da interface web, que implementa autenticação JWT. O token JWT contém informações sobre o usuário e suas permissões, sendo validado em cada requisição ao backend.

Processamento de Requisições: Requisições HTTP/HTTPS são recebidas pelo ALB, que as distribui para instâncias saudáveis da aplicação. A aplicação Spring Boot processa as requisições, implementa validações de negócio e interage com o banco de dados conforme necessário.

Armazenamento de Mídia: Fotos e vídeos capturados durante inspeções são enviados diretamente para o S3 através de URLs pré-assinadas, garantindo upload seguro sem passar pelo backend. Metadados dos arquivos são armazenados no banco de dados PostgreSQL.

Distribuição de Conteúdo: Arquivos de mídia são servidos através do CloudFront, que implementa cache global para reduzir latência e custos de transferência de dados.

2.4 Monitoramento e Observabilidade

Amazon CloudWatch: Coleta métricas de todos os componentes da infraestrutura e da aplicação. Dashboards customizados fornecem visibilidade em tempo real sobre a saúde do sistema.

CloudWatch Logs: Centraliza logs de todos os componentes, incluindo logs da aplicação, logs do ALB e logs do RDS. Log groups são configurados com retenção apropriada para cada tipo de log.

CloudWatch Alarms: Alertas automáticos são configurados para métricas críticas como CPU, memória, latência e taxa de erro. Notificações são enviadas via SNS para email ou integração com sistemas de ticketing.

AWS X-Ray: Implementa distributed tracing para rastreamento de requisições através de múltiplos serviços, facilitando a identificação de gargalos e problemas de performance.

3. Pré-requisitos e Preparação do Ambiente

3.1 Requisitos de Infraestrutura

Antes de iniciar o processo de implantação do Sistema de Inspeção de Equipamentos da Suzano, é fundamental garantir que todos os pré-requisitos de infraestrutura estejam atendidos. Esta seção detalha os requisitos técnicos, permissões necessárias e configurações preliminares que devem ser estabelecidas.

3.1.1 Conta AWS e Permissões

Conta AWS Ativa: É necessária uma conta AWS ativa com acesso aos serviços utilizados pelo sistema. A conta deve ter limites de serviço adequados para suportar a infraestrutura planejada. Recomenda-se verificar os limites atuais através do AWS Service Quotas console e solicitar aumentos se necessário.

Permissões IAM: O usuário ou role utilizado para o deployment deve possuir permissões administrativas ou, no mínimo, as seguintes permissões específicas: - EC2: Criação e gerenciamento de VPC, subnets, security groups, internet gateways e NAT gateways - ECS: Criação e gerenciamento de clusters, services, task definitions e capacity providers - RDS: Criação e gerenciamento de instâncias, subnet groups, parameter groups e snapshots - S3: Criação e gerenciamento de buckets, políticas de bucket e configurações de lifecycle - IAM: Criação e gerenciamento de roles, policies e instance profiles - CloudWatch: Criação de log groups, métricas customizadas e alarms - Route53: Gerenciamento de hosted zones e records (se domínio customizado for utilizado) - Certificate Manager: Solicitação e validação de certificados SSL - Secrets Manager: Criação e gerenciamento de secrets

Billing e Cost Management: Configurar alertas de billing para monitorar custos e evitar surpresas. Recomenda-se estabelecer budgets mensais e alertas quando 50%,

80% e 100% do orçamento for atingido.

3.1.2 Ferramentas de Desenvolvimento

Terraform: Versão 1.0 ou superior deve estar instalada na máquina de deployment. Terraform é utilizado para provisionar toda a infraestrutura como código, garantindo consistência e reprodutibilidade. A instalação pode ser realizada através do package manager do sistema operacional ou download direto do site oficial.

AWS CLI: Versão 2.0 ou superior deve estar instalada e configurada com credenciais apropriadas. O AWS CLI é utilizado para interações diretas com serviços AWS durante o processo de deployment e operações posteriores. A configuração deve incluir região padrão e formato de output preferido.

Docker: Versão 20.0 ou superior é necessária para build e push das imagens de container. Docker é utilizado para containerizar a aplicação backend e garantir consistência entre ambientes de desenvolvimento, teste e produção.

Java Development Kit (JDK): JDK 17 ou superior é necessário para compilar a aplicação Spring Boot. Recomenda-se utilizar OpenJDK ou Oracle JDK, ambos compatíveis com o projeto.

Maven: Versão 3.6 ou superior é utilizada para gerenciamento de dependências e build da aplicação Java. Maven deve estar configurado com acesso aos repositórios necessários e configurações de proxy se aplicável.

Git: Sistema de controle de versão para gerenciar o código fonte. Recomenda-se configurar SSH keys para acesso seguro aos repositórios.

3.1.3 Configurações de Rede

Conectividade Internet: Conexão estável com a internet é necessária para download de dependências, push de imagens Docker e comunicação com APIs da AWS. Recomenda-se largura de banda mínima de 10 Mbps para upload.

Configurações de Proxy: Se a organização utiliza proxy corporativo, as ferramentas devem ser configuradas adequadamente: - Docker: Configurar proxy settings no daemon Docker - Maven: Configurar proxy no arquivo settings.xml - AWS CLI: Configurar variáveis de ambiente HTTP_PROXY e HTTPS_PROXY - Terraform: Configurar variáveis de ambiente para proxy se necessário

DNS e Domínio: Se um domínio customizado será utilizado, é necessário: - Domínio registrado e acessível - Acesso ao gerenciamento de DNS do domínio - Capacidade de criar registros CNAME ou delegar subdomínio para Route53

3.2 Configuração do Ambiente de Desenvolvimento

3.2.1 Configuração do AWS CLI

A configuração adequada do AWS CLI é fundamental para o sucesso do deployment. O processo envolve a configuração de credenciais, região padrão e perfis se necessário.

```
# Configuração básica do AWS CLI
aws configure

# Verificação da configuração
aws sts get-caller-identity

# Configuração de perfil específico (opcional)
aws configure --profile suzano-prod
```

As credenciais podem ser configuradas através de múltiplos métodos: - Access Keys: Método tradicional usando Access Key ID e Secret Access Key - IAM Roles: Recomendado para instâncias EC2 ou ambientes corporativos - AWS SSO: Para organizações que utilizam Single Sign-On - Temporary Credentials: Para acesso temporário através de STS

3.2.2 Configuração do Terraform

Terraform requer configuração inicial para gerenciar o estado da infraestrutura de forma segura e colaborativa.

Backend Configuration: O estado do Terraform deve ser armazenado remotamente para permitir colaboração e garantir consistência:

```
terraform {
  backend "s3" {
    bucket      = "suzano-terraform-state"
    key         = "inspection-system/terraform.tfstate"
    region      = "us-east-1"
    dynamodb_table = "terraform-locks"
    encrypt     = true
  }
}
```

Provider Configuration: O provider AWS deve ser configurado com a região apropriada e tags padrão:

```
provider "aws" {  
  region = "us-east-1"  
  
  default_tags {  
    tags = {  
      Project      = "Suzano-Inspection-System"  
      Environment = "production"  
      ManagedBy   = "Terraform"  
    }  
  }  
}
```

3.2.3 Configuração do Docker

Docker deve ser configurado para build e push de imagens para o Amazon ECR:

```
# Login no ECR  
aws ecr get-login-password --region us-east-1 | docker login --username AWS --  
password-stdin <account-id>.dkr.ecr.us-east-1.amazonaws.com  
  
# Verificação da configuração  
docker info
```

3.3 Preparação dos Recursos AWS

3.3.1 Criação do Backend do Terraform

Antes de executar o Terraform, é necessário criar os recursos para armazenamento do estado:

S3 Bucket para Estado: Bucket S3 com versionamento habilitado para armazenar o arquivo de estado do Terraform:

```
# Criar bucket para estado do Terraform
aws s3 mb s3://suzano-terraform-state-$(date +%s) --region us-east-1

# Habilitar versionamento
aws s3api put-bucket-versioning \
  --bucket suzano-terraform-state-$(date +%s) \
  --versioning-configuration Status=Enabled

# Habilitar criptografia
aws s3api put-bucket-encryption \
  --bucket suzano-terraform-state-$(date +%s) \
  --server-side-encryption-configuration '{
    "Rules": [{
      "ApplyServerSideEncryptionByDefault": {
        "SSEAlgorithm": "AES256"
      }
    }]
  }'
```

DynamoDB Table para Locks: Tabela DynamoDB para implementar locking do estado do Terraform:

```
# Criar tabela para locks
aws dynamodb create-table \
  --table-name terraform-locks \
  --attribute-definitions AttributeName=LockID,AttributeType=S \
  --key-schema AttributeName=LockID,KeyType=HASH \
  --provisioned-throughput ReadCapacityUnits=5,WriteCapacityUnits=5 \
  --region us-east-1
```

3.3.2 Configuração de Domínio (Opcional)

Se um domínio customizado será utilizado, é necessário configurar o Route53:

Hosted Zone: Criar ou configurar hosted zone no Route53 para o domínio:

```
# Criar hosted zone
aws route53 create-hosted-zone \
  --name inspection.suzano.com \
  --caller-reference $(date +%s)
```

Delegação de DNS: Configurar os name servers do domínio para apontar para o Route53.

3.4 Validação dos Pré-requisitos

Antes de prosseguir com o deployment, é importante validar que todos os pré-requisitos foram atendidos:

3.4.1 Script de Validação

Um script de validação pode ser utilizado para verificar automaticamente os pré-requisitos:

```
#!/bin/bash

echo "Validando pré-requisitos para deployment..."

# Verificar AWS CLI
if ! command -v aws &> /dev/null; then
    echo "❌ AWS CLI não encontrado"
    exit 1
else
    echo "✅ AWS CLI encontrado: $(aws --version)"
fi

# Verificar credenciais AWS
if ! aws sts get-caller-identity &> /dev/null; then
    echo "❌ Credenciais AWS não configuradas"
    exit 1
else
    echo "✅ Credenciais AWS válidas"
fi

# Verificar Terraform
if ! command -v terraform &> /dev/null; then
    echo "❌ Terraform não encontrado"
    exit 1
else
    echo "✅ Terraform encontrado: $(terraform version)"
fi

# Verificar Docker
if ! command -v docker &> /dev/null; then
    echo "❌ Docker não encontrado"
    exit 1
else
    echo "✅ Docker encontrado: $(docker --version)"
fi

# Verificar Java
if ! command -v java &> /dev/null; then
    echo "❌ Java não encontrado"
    exit 1
else
    echo "✅ Java encontrado: $(java -version 2>&1 | head -n 1)"
fi

# Verificar Maven
if ! command -v mvn &> /dev/null; then
    echo "❌ Maven não encontrado"
    exit 1
else
    echo "✅ Maven encontrado: $(mvn -version | head -n 1)"
fi

echo "✅ Todos os pré-requisitos foram atendidos!"
```

3.4.2 Verificação de Limites de Serviço

Verificar os limites atuais dos serviços AWS que serão utilizados:

```
# Verificar limites do EC2
aws service-quotas get-service-quota \
  --service-code ec2 \
  --quota-code L-1216C47A # VPCs per Region

# Verificar limites do RDS
aws service-quotas get-service-quota \
  --service-code rds \
  --quota-code L-7B6409FD # DB instances

# Verificar limites do ECS
aws service-quotas get-service-quota \
  --service-code ecs \
  --quota-code L-21C621EB # Clusters per account
```

3.5 Considerações de Segurança

3.5.1 Princípio do Menor Privilégio

Todas as permissões devem seguir o princípio do menor privilégio, concedendo apenas as permissões mínimas necessárias para cada componente funcionar adequadamente.

3.5.2 Criptografia

Todos os dados devem ser criptografados em repouso e em trânsito: - RDS: Criptografia habilitada com chaves KMS - S3: Criptografia server-side habilitada - ECS: Secrets gerenciados pelo AWS Secrets Manager - ALB: Certificados SSL/TLS obrigatórios

3.5.3 Auditoria e Compliance

Configurar AWS CloudTrail para auditoria de todas as ações realizadas na conta AWS. Habilitar AWS Config para monitoramento de compliance de recursos.

4. Processo de Implantação

4.1 Visão Geral do Processo

O processo de implantação do Sistema de Inspeção de Equipamentos da Suzano foi projetado para ser automatizado, reproduzível e seguro. Utilizando Infrastructure as Code (IaC) através do Terraform e containerização com Docker, o deployment garante consistência entre diferentes ambientes e facilita a manutenção e atualizações futuras.

O processo completo de implantação é dividido em cinco fases principais: preparação do ambiente, provisionamento da infraestrutura, build e deploy da aplicação, configuração inicial e validação do sistema. Cada fase possui checkpoints específicos que devem ser validados antes de prosseguir para a próxima etapa.

4.2 Fase 1: Preparação do Ambiente

4.2.1 Clonagem do Repositório

O primeiro passo é obter o código fonte completo do sistema:

```
# Clonar o repositório principal
git clone https://github.com/suzano/inspection-system.git
cd inspection-system

# Verificar a estrutura do projeto
tree -L 2
```

A estrutura do projeto deve conter os seguintes diretórios principais: - `inspection-backend/` : Código fonte da aplicação Java Spring Boot - `suzano-inspection-web/` : Interface web React - `aws-infrastructure/` : Configurações Terraform e scripts de deployment - `docs/` : Documentação técnica e manuais

4.2.2 Configuração de Variáveis de Ambiente

Criar arquivo de configuração com as variáveis específicas do ambiente:

```
# Criar arquivo de configuração
cp aws-infrastructure/terraform/terraform.tfvars.example aws-
infrastructure/terraform/terraform.tfvars

# Editar configurações específicas
vim aws-infrastructure/terraform/terraform.tfvars
```

Exemplo de configuração para ambiente de produção:

```
# Configurações básicas
aws_region = "us-east-1"
environment = "prod"
project_name = "suzano-inspection"

# Configurações de rede
vpc_cidr = "10.0.0.0/16"
public_subnet_cidrs = ["10.0.1.0/24", "10.0.2.0/24"]
private_subnet_cidrs = ["10.0.10.0/24", "10.0.20.0/24"]

# Configurações de banco de dados
db_instance_class = "db.t3.medium"
db_allocated_storage = 100
multi_az = true
backup_retention_period = 30

# Configurações de aplicação
ecs_task_cpu = 1024
ecs_task_memory = 2048
desired_count = 3

# Configurações de domínio (opcional)
domain_name = "inspection.suzano.com"
notification_email = "devops@suzano.com.br"

# Configurações de segurança
enable_waf = true
enable_deletion_protection = true
```

4.3 Fase 2: Provisionamento da Infraestrutura

4.3.1 Inicialização do Terraform

```
cd aws-infrastructure/terraform

# Inicializar Terraform
terraform init

# Validar configuração
terraform validate

# Planejar deployment
terraform plan -out=tfplan
```


O comando `terraform plan` deve exibir um resumo dos recursos que serão criados. Para um deployment completo, espera-se a criação de aproximadamente 50-60 recursos AWS, incluindo: - 1 VPC com 6 subnets - 2 NAT Gateways e 1 Internet Gateway - 1 Application Load Balancer - 1 ECS Cluster com Service e Task Definition - 1 RDS PostgreSQL instance - 3 S3 Buckets - 1 CloudFront Distribution - Múltiplos Security Groups, IAM Roles e CloudWatch resources

4.3.2 Execução do Deployment

```
# Aplicar configuração
terraform apply tfplan

# Aguardar conclusão (aproximadamente 15-20 minutos)
# Salvar outputs importantes
terraform output -json > ../outputs.json
```

Durante a execução, o Terraform criará os recursos na seguinte ordem: 1. Recursos de rede (VPC, subnets, gateways) 2. Recursos de segurança (Security Groups, IAM Roles) 3. Recursos de armazenamento (S3, RDS) 4. Recursos de computação (ECS, ALB) 5. Recursos de monitoramento (CloudWatch, SNS)

4.3.3 Validação da Infraestrutura

Após a conclusão do Terraform, validar que todos os recursos foram criados corretamente:

```
# Verificar status do cluster ECS
aws ecs describe-clusters --clusters suzano-inspection-prod-cluster

# Verificar status do RDS
aws rds describe-db-instances --db-instance-identifier suzano-inspection-prod-database

# Verificar status do ALB
aws elbv2 describe-load-balancers --names suzano-inspection-prod-alb

# Verificar buckets S3
aws s3 ls | grep suzano-inspection
```

4.4 Fase 3: Build e Deploy da Aplicação

4.4.1 Build da Aplicação Backend

```
cd ../../inspection-backend

# Compilar aplicação
mvn clean package -DskipTests

# Verificar se o JAR foi criado
ls -la target/*.jar
```

O processo de build deve gerar um arquivo JAR executável no diretório `target/`. Este arquivo contém toda a aplicação Spring Boot e suas dependências.

4.4.2 Containerização

```
# Obter URL do repositório ECR
ECR_REPO=$(cat ../aws-infrastructure/outputs.json | jq -r
'.ecr_repository_url.value')

# Build da imagem Docker
docker build -t suzano-inspection-app .

# Tag para ECR
docker tag suzano-inspection-app:latest $ECR_REPO:latest

# Login no ECR
aws ecr get-login-password --region us-east-1 | docker login --username AWS --
password-stdin $ECR_REPO

# Push para ECR
docker push $ECR_REPO:latest
```

4.4.3 Deploy no ECS

```
# Obter informações do cluster
CLUSTER_NAME=$(cat ../aws-infrastructure/outputs.json | jq -r
'.ecs_cluster_name.value')
SERVICE_NAME=$(cat ../aws-infrastructure/outputs.json | jq -r
'.ecs_service_name.value')

# Forçar novo deployment
aws ecs update-service \
  --cluster $CLUSTER_NAME \
  --service $SERVICE_NAME \
  --force-new-deployment

# Aguardar estabilização do serviço
aws ecs wait services-stable \
  --cluster $CLUSTER_NAME \
  --services $SERVICE_NAME
```

4.5 Fase 4: Configuração Inicial

4.5.1 Configuração do Banco de Dados

```
# Obter endpoint do banco
DB_ENDPOINT=$(cat ../aws-infrastructure/outputs.json | jq -r
'.rds_endpoint.value')

# Obter credenciais do Secrets Manager
SECRET_ARN=$(cat ../aws-infrastructure/outputs.json | jq -r
'.db_secret_arn.value')
DB_CREDENTIALS=$(aws secretsmanager get-secret-value --secret-id `SECRET_ARN
--query SecretString --output text)

# Extrair usuário e senha
DB_USER=$(echo `DB_CREDENTIALS | jq -r '.username')
DB_PASS=$(echo `DB_CREDENTIALS | jq -r '.password')

# Conectar ao banco e executar scripts de inicialização
psql -h `DB_ENDPOINT -U `DB_USER -d suzano_inspection -f scripts/init-
database.sql
```

4.5.2 Configuração de Dados Iniciais

```
# Executar scripts de dados iniciais
psql -h `DB_ENDPOINT -U `DB_USER -d suzano_inspection -f scripts/seed-
data.sql

# Verificar se os dados foram inseridos
psql -h `DB_ENDPOINT -U `DB_USER -d suzano_inspection -c "SELECT COUNT(*)
FROM users;"
```

4.6 Fase 5: Validação do Sistema

4.6.1 Testes de Conectividade

```
# Obter URL da aplicação
APP_URL=$(cat ../aws-infrastructure/outputs.json | jq -r
'.application_url.value')

# Testar health check
curl -f $APP_URL/actuator/health

# Testar endpoint de login
curl -X POST $APP_URL/api/auth/login \
-H "Content-Type: application/json" \
-d '{"username": "admin", "password": "admin123"}'
```

4.6.2 Testes de Funcionalidade

```
# Executar testes automatizados
cd ../inspection-backend
mvn test -Dspring.profiles.active=integration

# Executar testes de carga (opcional)
cd ../load-tests
./run-load-tests.sh $APP_URL
```

4.6.3 Validação de Monitoramento

```
# Verificar métricas no CloudWatch
aws cloudwatch get-metric-statistics \
  --namespace AWS/ECS \
  --metric-name CPUUtilization \
  --dimensions Name=ServiceName,Value=$SERVICE_NAME \
  Name=ClusterName,Value=$CLUSTER_NAME \
  --start-time $(date -u -d '1 hour ago' +%Y-%m-%dT%H:%M:%S) \
  --end-time $(date -u +%Y-%m-%dT%H:%M:%S) \
  --period 300 \
  --statistics Average

# Verificar logs da aplicação
aws logs describe-log-groups --log-group-name-prefix /aws/ecs/suzano-inspection
```

4.7 Troubleshooting Comum

4.7.1 Problemas de Terraform

Erro de Permissões:

```
# Verificar permissões do usuário atual
aws iam get-user
aws iam list-attached-user-policies --user-name <username>

# Verificar se o usuário tem as permissões necessárias
aws iam simulate-principal-policy \
  --policy-source-arn $(aws sts get-caller-identity --query Arn --output text) \
  --action-names ec2:CreateVpc \
  --resource-arns "*" "
```

Erro de Limites de Serviço:

```
# Verificar limites atuais
aws service-quotas get-service-quota \
  --service-code ec2 \
  --quota-code L-F678F1CE # Running On-Demand instances

# Solicitar aumento de limite se necessário
aws service-quotas request-service-quota-increase \
  --service-code ec2 \
  --quota-code L-F678F1CE \
  --desired-value 20
```

4.7.2 Problemas de ECS

Container não inicia:

```
# Verificar logs do container
aws logs get-log-events \
  --log-group-name /aws/ecs/suzano-inspection-prod/app \
  --log-stream-name ecs/app/$(aws ecs list-tasks --cluster $CLUSTER_NAME --
service-name $SERVICE_NAME --query 'taskArns[0]' --output text | cut -d'/' -f3)

# Verificar task definition
aws ecs describe-task-definition \
  --task-definition suzano-inspection-prod-app
```

Health Check falhando:

```
# Verificar configuração do target group
aws elbv2 describe-target-health \
  --target-group-arn $(aws elbv2 describe-target-groups --names suzano-
inspection-prod-app-tg --query 'TargetGroups[0].TargetGroupArn' --output text)

# Testar health check manualmente
curl -f $APP_URL/actuator/health
```

4.7.3 Problemas de RDS

Conexão recusada:

```
# Verificar security groups
aws ec2 describe-security-groups \
  --filters Name=group-name,Values=*rds* \
  --query 'SecurityGroups[*].
{GroupId:GroupId,GroupName:GroupName,InboundRules:IpPermissions}'

# Verificar se o RDS está acessível
telnet $DB_ENDPOINT 5432
```

4.8 Rollback e Recovery

4.8.1 Rollback da Aplicação

```
# Listar versões anteriores da imagem
aws ecr describe-images \
  --repository-name suzano-inspection-prod-app \
  --query 'imageDetails[*].{ImageTags:imageTags,PushedAt:imagePushedAt}' \
  --output table

# Fazer rollback para versão anterior
aws ecs update-service \
  --cluster $CLUSTER_NAME \
  --service $SERVICE_NAME \
  --task-definition suzano-inspection-prod-app:PREVIOUS_REVISION
```

4.8.2 Rollback da Infraestrutura

```
# Verificar histórico do estado do Terraform
aws s3api list-object-versions \
  --bucket suzano-terraform-state \
  --prefix terraform.tfstate

# Restaurar versão anterior do estado (se necessário)
aws s3api get-object \
  --bucket suzano-terraform-state \
  --key terraform.tfstate \
  --version-id <version-id> \
  terraform.tfstate.backup

# Aplicar configuração anterior
terraform apply -auto-approve
```

5. Operação e Monitoramento

5.1 Monitoramento da Aplicação

O Sistema de Inspeção de Equipamentos da Suzano implementa uma estratégia abrangente de monitoramento que combina métricas de infraestrutura, métricas de aplicação e logs centralizados. Esta abordagem multicamada garante visibilidade completa sobre a saúde e performance do sistema.

5.1.1 Dashboard Principal

O dashboard principal do CloudWatch fornece uma visão consolidada de todas as métricas críticas do sistema. Acessível através da URL fornecida nos outputs do

Terraform, o dashboard é organizado em seções que correspondem aos diferentes componentes da arquitetura.

Métricas de ECS: A seção superior do dashboard exibe métricas relacionadas ao cluster ECS, incluindo utilização de CPU e memória dos containers, número de tasks em execução e taxa de restart de containers. Estas métricas são fundamentais para identificar problemas de performance e necessidades de scaling.

Métricas de RDS: A seção central monitora a saúde do banco de dados PostgreSQL, incluindo utilização de CPU, conexões ativas, espaço livre em disco e latência de queries. Alertas automáticos são configurados para situações que requerem atenção imediata.

Métricas de ALB: A seção inferior foca no Application Load Balancer, monitorando número de requisições, tempo de resposta, códigos de status HTTP e distribuição de tráfego entre targets. Estas métricas são essenciais para identificar problemas de conectividade e performance.

5.1.2 Alertas Automáticos

O sistema implementa um conjunto abrangente de alertas automáticos configurados para detectar anomalias e situações críticas:

Alertas de Performance: - CPU do ECS acima de 80% por mais de 10 minutos - Memória do ECS acima de 85% por mais de 10 minutos - Tempo de resposta do ALB acima de 1 segundo por mais de 5 minutos - CPU do RDS acima de 75% por mais de 15 minutos

Alertas de Disponibilidade: - Mais de 10 erros 5XX no ALB em 5 minutos - Falha no health check do ECS por mais de 3 tentativas consecutivas - Espaço livre no RDS abaixo de 2GB - Número de conexões no RDS acima de 80% do limite

Alertas de Segurança: - Tentativas de login falhadas acima de 100 em 5 minutos - Acesso negado (403) acima de 50 em 5 minutos - Padrões suspeitos de tráfego detectados pelo WAF

5.1.3 Logs Centralizados

Todos os logs do sistema são centralizados no CloudWatch Logs, organizados em grupos lógicos que facilitam a busca e análise:

Application Logs: Logs da aplicação Spring Boot incluem informações sobre requisições HTTP, operações de banco de dados, erros de aplicação e eventos de negócio. O nível de log pode ser ajustado dinamicamente através de variáveis de ambiente.

Infrastructure Logs: Logs de infraestrutura incluem logs do ALB (access logs), logs do ECS (container logs) e logs do RDS (slow query logs). Estes logs são essenciais para troubleshooting de problemas de infraestrutura.

Security Logs: Logs de segurança incluem eventos do WAF, tentativas de autenticação e ações administrativas. Estes logs são retidos por períodos mais longos para atender requisitos de compliance.

5.2 Procedimentos Operacionais

5.2.1 Deployment de Novas Versões

O processo de deployment de novas versões foi projetado para minimizar downtime e garantir rollback rápido em caso de problemas:


```

# 1. Build da nova versão
cd inspection-backend
mvn clean package -DskipTests

# 2. Build e push da imagem Docker
docker build -t suzano-inspection-app:v2.1.0 .
docker tag suzano-inspection-app:v2.1.0 $ECR_REPO:v2.1.0
docker push $ECR_REPO:v2.1.0

# 3. Atualizar task definition
aws ecs register-task-definition \
  --family suzano-inspection-prod-app \
  --task-role-arn $TASK_ROLE_ARN \
  --execution-role-arn $EXECUTION_ROLE_ARN \
  --network-mode awsvpc \
  --requires-compatibilities FARGATE \
  --cpu 1024 \
  --memory 2048 \
  --container-definitions file://container-definitions.json

# 4. Atualizar serviço ECS
aws ecs update-service \
  --cluster $CLUSTER_NAME \
  --service $SERVICE_NAME \
  --task-definition suzano-inspection-prod-app:LATEST

# 5. Monitorar deployment
aws ecs wait services-stable \
  --cluster $CLUSTER_NAME \
  --services $SERVICE_NAME

```

5.2.2 Scaling Manual

Em situações de pico de demanda, o sistema pode ser escalado manualmente:

```

# Escalar horizontalmente (aumentar número de tasks)
aws ecs update-service \
  --cluster $CLUSTER_NAME \
  --service $SERVICE_NAME \
  --desired-count 5

# Escalar verticalmente (aumentar recursos por task)
# Requer atualização da task definition com novos valores de CPU/memória
aws ecs register-task-definition \
  --family suzano-inspection-prod-app \
  --cpu 2048 \
  --memory 4096 \
  --container-definitions file://container-definitions-large.json

```

5.2.3 Backup e Restore

Backup Automático do RDS: O RDS está configurado com backups automáticos diários com retenção de 30 dias. Backups manuais podem ser criados conforme

necessário:

```
# Criar snapshot manual
aws rds create-db-snapshot \
  --db-instance-identifier suzano-inspection-prod-database \
  --db-snapshot-identifier manual-snapshot-$(date +%Y%m%d-%H%M%S)

# Listar snapshots disponíveis
aws rds describe-db-snapshots \
  --db-instance-identifier suzano-inspection-prod-database
```

Backup de Arquivos S3: Arquivos de mídia no S3 são automaticamente versionados e têm lifecycle policies configuradas:

```
# Verificar versionamento
aws s3api get-bucket-versioning \
  --bucket $MEDIA_BUCKET_NAME

# Listar versões de um objeto
aws s3api list-object-versions \
  --bucket $MEDIA_BUCKET_NAME \
  --prefix inspections/2024/
```

5.3 Manutenção Preventiva

5.3.1 Atualizações de Segurança

Patches do Sistema Operacional: Como o sistema utiliza Fargate, os patches do sistema operacional são gerenciados automaticamente pela AWS. No entanto, é importante manter as imagens base Docker atualizadas:

```
# Atualizar imagem base no Dockerfile
FROM openjdk:17-jre-slim

# Rebuild e redeploy regularmente
docker build --no-cache -t suzano-inspection-app:latest .
```

Atualizações de Dependências: Dependências Java devem ser atualizadas regularmente para corrigir vulnerabilidades:

```
# Verificar vulnerabilidades conhecidas
mvn org.owasp:dependency-check-maven:check

# Atualizar dependências
mvn versions:use-latest-versions
mvn versions:update-properties
```

5.3.2 Otimização de Performance

Análise de Performance do Banco: Utilizar Performance Insights do RDS para identificar queries lentas:

```
# Obter top queries por tempo de execução
aws pi get-resource-metrics \
  --service-type RDS \
  --identifier $DB_RESOURCE_ID \
  --metric-queries file://performance-queries.json \
  --start-time $(date -u -d '1 day ago' +%Y-%m-%dT%H:%M:%S) \
  --end-time $(date -u +%Y-%m-%dT%H:%M:%S)
```

Otimização de Custos: Revisar regularmente os custos e otimizar recursos subutilizados:

```
# Analisar custos por serviço
aws ce get-cost-and-usage \
  --time-period Start=$(date -d '1 month ago' +%Y-%m-%d),End=$(date +%Y-%m-%d) \
  --granularity MONTHLY \
  --metrics BlendedCost \
  --group-by Type=DIMENSION,Key=SERVICE
```

5.4 Disaster Recovery

5.4.1 Estratégia de DR

O sistema implementa uma estratégia de disaster recovery baseada em múltiplas zonas de disponibilidade e backups automatizados:

RTO (Recovery Time Objective): 4 horas **RPO (Recovery Point Objective):** 1 hora

5.4.2 Procedimentos de Recovery

Falha de Zona de Disponibilidade: O sistema é automaticamente resiliente a falhas de AZ devido à configuração Multi-AZ do RDS e distribuição de tasks ECS:

```
# Verificar distribuição de tasks por AZ
aws ecs describe-tasks \
  --cluster $CLUSTER_NAME \
  --tasks $(aws ecs list-tasks --cluster $CLUSTER_NAME --service-name $SERVICE_NAME --query 'taskArns' --output text)
```

Falha Regional: Em caso de falha regional completa, o sistema pode ser recriado em outra região:

```
# 1. Restaurar RDS em nova região a partir de snapshot
aws rds restore-db-instance-from-db-snapshot \
  --db-instance-identifier suzano-inspection-dr-database \
  --db-snapshot-identifier arn:aws:rds:us-east-1:account:snapshot:snapshot-name \
  --region us-west-2

# 2. Replicar dados S3 para nova região
aws s3 sync s3://$MEDIA_BUCKET_NAME s3://$DR_MEDIA_BUCKET_NAME --region us-west-2

# 3. Executar Terraform em nova região
cd aws-infrastructure/terraform
terraform workspace new dr
terraform apply -var="aws_region=us-west-2"
```

6. Segurança e Compliance

6.1 Controles de Segurança Implementados

6.1.1 Segurança de Rede

Isolamento de Rede: A VPC dedicada com sub-redes privadas garante que componentes críticos como banco de dados e aplicação não sejam diretamente acessíveis da internet. Todo o tráfego de entrada passa pelo Application Load Balancer, que atua como um ponto de controle centralizado.

Segmentação: Security Groups implementam micro-segmentação, permitindo apenas o tráfego necessário entre componentes. Por exemplo, o banco de dados aceita conexões apenas dos containers da aplicação na porta 5432.

Proteção DDoS: AWS Shield Standard fornece proteção automática contra ataques DDoS. O WAF adiciona uma camada extra de proteção contra ataques de aplicação web.

6.1.2 Criptografia

Em Trânsito: Todo o tráfego HTTP é redirecionado para HTTPS. Certificados SSL/TLS são gerenciados automaticamente pelo AWS Certificate Manager. Conexões entre componentes internos utilizam criptografia quando suportada.

Em Repouso: Todos os dados são criptografados em repouso: - RDS: Criptografia habilitada com chaves AWS KMS - S3: Server-side encryption com AES-256 - EBS: Volumes criptografados automaticamente - Secrets Manager: Secrets criptografados com chaves KMS

6.1.3 Gestão de Identidade e Acesso

Princípio do Menor Privilégio: Todas as IAM roles seguem o princípio do menor privilégio, concedendo apenas as permissões mínimas necessárias.

Rotação de Credenciais: Senhas do banco de dados são armazenadas no Secrets Manager com rotação automática configurada para 30 dias.

Auditoria: AWS CloudTrail registra todas as ações administrativas para auditoria e compliance.

6.2 Compliance e Regulamentações

6.2.1 LGPD (Lei Geral de Proteção de Dados)

O sistema implementa controles específicos para atender aos requisitos da LGPD:

Minimização de Dados: Apenas dados necessários para as operações de inspeção são coletados e armazenados.

Consentimento: Interface de usuário inclui mecanismos para obtenção e gerenciamento de consentimento quando aplicável.

Direito ao Esquecimento: APIs implementam funcionalidades para exclusão de dados pessoais quando solicitado.

Portabilidade: Sistema permite exportação de dados em formatos estruturados.

6.2.2 Normas Industriais

ISO 27001: Controles de segurança implementados seguem as diretrizes da ISO 27001 para gestão de segurança da informação.

SOC 2: Infraestrutura AWS atende aos requisitos SOC 2 Type II, fornecendo garantias sobre controles de segurança, disponibilidade e confidencialidade.

7. Custos e Otimização

7.1 Estimativa de Custos

7.1.1 Custos Mensais Estimados (Região us-east-1)

Serviço	Configuração	Custo Mensal (USD)
ECS Fargate	3 tasks, 1 vCPU, 2GB RAM	\$65
RDS PostgreSQL	db.t3.medium, Multi-AZ	\$85
Application Load Balancer	1 ALB	\$23
S3 Standard	100GB armazenamento	\$2.30
CloudFront	1TB transferência	\$85
NAT Gateway	2 gateways	\$90
CloudWatch	Logs e métricas	\$15
Secrets Manager	5 secrets	\$2.50
Total Estimado		\$367.80

7.1.2 Estratégias de Otimização

Reserved Instances: Para ambientes de produção estáveis, considerar Reserved Instances para RDS pode reduzir custos em até 40%.

S3 Intelligent Tiering: Configurar Intelligent Tiering para arquivos de mídia pode reduzir custos de armazenamento em 20-30%.

CloudWatch Logs Retention: Ajustar períodos de retenção de logs baseado em requisitos de compliance pode reduzir custos de armazenamento.

Auto Scaling: Configurar auto scaling agressivo para reduzir custos durante períodos de baixa demanda.

7.2 Monitoramento de Custos

```
# Configurar alertas de billing
aws budgets create-budget \
  --account-id $(aws sts get-caller-identity --query Account --output text) \
  --budget file://budget-config.json \
  --notifications-with-subscribers file://budget-notifications.json
```

8. Conclusão

O Sistema de Inspeção de Equipamentos da Suzano representa uma solução tecnológica moderna e robusta que atende aos desafios identificados no Desafio de Inovação 2025. A arquitetura cloud-native baseada em AWS garante escalabilidade, confiabilidade e segurança, enquanto a automação completa do deployment facilita a manutenção e atualizações futuras.

A implementação bem-sucedida deste sistema proporcionará benefícios significativos para a Suzano, incluindo melhoria na qualidade dos dados de inspeção, redução de erros operacionais, otimização de processos de manutenção e criação de uma base sólida para análises preditivas e tomada de decisões baseada em dados.

O guia de implantação fornecido garante que a equipe técnica da Suzano possa deployar, operar e manter o sistema de forma eficiente, seguindo as melhores práticas de DevOps e segurança. O monitoramento abrangente e os procedimentos operacionais detalhados asseguram que o sistema mantenha alta disponibilidade e performance ao longo do tempo.

Documento preparado por: Manus AI

Data: Junho 2025

Versão: 1.0

Próxima revisão: Dezembro 2025